Meeting 8 SP2023

Optimized Encryption for Flexible Processors

@pradyun

Outline

Background - Flexible Electronics

Background - Encryption

The Actual Paper Part

Section 1

Background - Flexible Electronics

Traditional Chip Fabrication

- Most modern digital ICs use CMOS
  - High-speed/advanced processors tend to use FinFET
  - CMOS is relatively outdated compared to non-planar technologies, but more cost-efficient
- Fabricated using crystalline silicon wafer/substrate
  - Forms the basis of most modern electronics

Drawbacks of Silicon-based CMOS logic

Let's look at this from a low-cost electronics perspective.

- Inherently high fabrication/processing cost
- Limitations on thinness, and flexibility
  - Lots of wearable/commodity applications that require these attributes

Enter Bendy Electronics

- Thin-Film Transistors (TFTs): Flexible substrates with active thin-film semiconductors
  - *Potential Substrates*: paper, plastic, metal foil
  - *Potential Semiconductors*: amorphous silicon, metal oxide
- Significally cheaper to produce, thinner profile, and bendy - fixes all our issues!

But Why Isn't My Arduino Bendy?

- Very *very* recent advance in fabrication techniques
  - First non-silicon processsor published last year!
- Similar challenges to old nMOS processors of days yore, as well as the drawbacks of just being new
- Drawbacks make flexible processors perfect for low-cost, high-yield applications but impractical for performance-first

Assumption Going Forward

A flexible processor is viable, and /exists

For the purposes of this paper, we'll be talking about IGZO TFT-based ICs.

Section 2

Background - Encryption

Encryption Standards

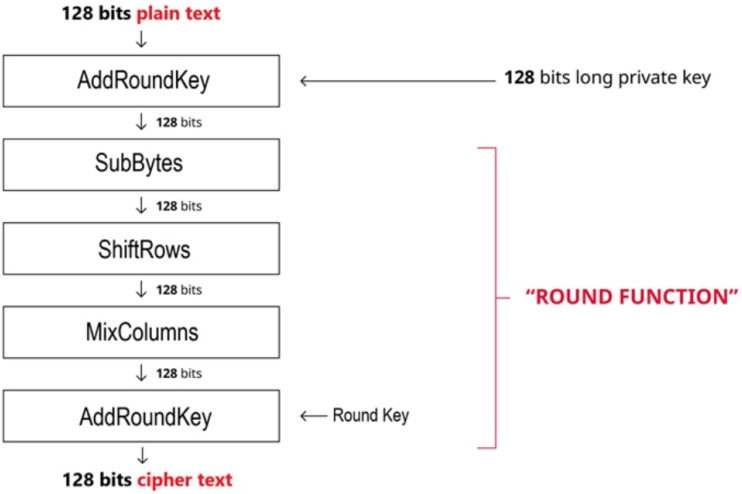
- With the high volume of data in today's society, it needs to be encrypted to ensure some degree of privacy
  - A common standard to do this is AES
  - There are other potentially better standards out there, but we can use this as a baseline

AES: A Basic Overview

- Symmetric block cipher - we have a key and and an input 'block'
  - Keys usually sized 128/192/256 bits, 128-bit block size. Longer key sizes increase computation time
- For encryption, we take our input data, separate it into 128-bit blocks, then separate that block into a 4x4 byte matrix

byte 00	byte 04	byte 08	byte 12
byte 01	byte 05	byte 09	byte 13
byte 02	byte 06	byte 10	byte 14
byte 03	byte 07	byte 11	byte 15

AES: Encryption Flowchart



Skimming past Case Studies

- Most existing hardware AES implementations try to optimize some of the following parameters
  - Integrate encryption/decryption to minimize area
  - Matrix multiplication w/ GFM
  - Better SBox implementation (AES-GCM algorithm)
    - Or just turn it into a LUT

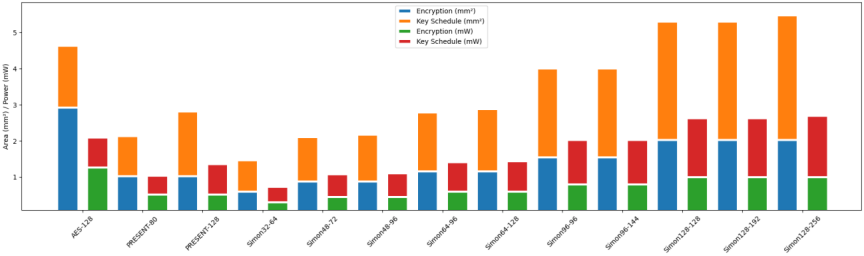
Section 3

The Actual Paper Part

Drawbacks of Current Topology

- Flexible/low-cost applications generally have limited power/area constraints
- High power consumption to generate keys and key schedule

Why Bother Optimizing this Step?



Offloading Key Generation

- Store a pre-computed key schedule in separate LPROM
  - This should be done when the chip is fabricated
- Number of keys you want to store will depend on lifetime

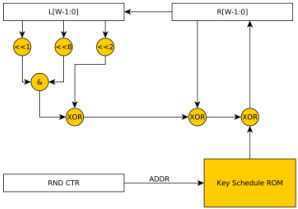
Cryptanalytic Attacks

- Minimum number of keys is determined by trying to prevent a birthday attack with relative confidence
  - Hint hint: Since low-cost processors don't stick around for long, the amount of data we try to prevent collisions within is (relatively) small
- Other attacks exist, but they usually require as much data as the birthday attack, if not more
- This also prevents potential hardware implementation-based vulnerabilities, since key generation/expansion is separated from the AES implementation itself

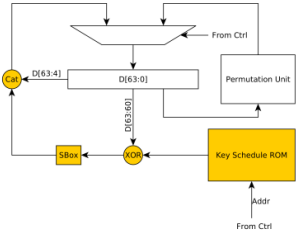
RIP Recycling

Applications	Lifetime	Sample Rate (Hz)	Precision (bit)	Throughput (bit s^{-1})	Keys 32 / 48 / 64 / 128 bit blocks
ECG sensor	12 - 48 Hours / Single Use	100	12	1200	2235 / 6 / 1 / 1
Blood Pressure Sensor	24 Hours / Single Use	60	8	480	447 / 2 / 1 / 1
Electronic Nose - body odor	Hours	-	-	-	- / - / -
Heart Rate Sensor	12 - 48 Hours / Single Use	0.8 - 3.2	1	2	4 / 1 / 1 / 1
Body Temp. Sensor	Single Use	0.1 - 1	8	4.4	1 / 1 / 1 / 1
Smart Bandage	days	1	10	10	47 / 1 / 1 / 1
Sensor for PD detection	13-16 hours per day 14-20 Days / Single Use	50 - 100	13	650	8069 / 21 / 2 / 1
Oral-Nasal Airflow	2-4 Hours	16 - 25	8	164	26 / 1 / 1 / 1
Perspiration Sensor	2 Hours / Single Use	25	8	200	16 / 1 / 1 / 1
Pedometer	1 - 4 hrs	20	13	260	41 / 1 / 1 / 1
Human activity recognition	1 - 4 hrs	30	8	240	38 / 1 / 1 / 1
EEG sensor	8 Hours / Single Use	100-200	8	1200	373 / 1 / 1 / 1
Temperature Sensor (IoT/Biomedical)	Days / Hours / Single Use	< 1	8	8	23 / 1 / 1 / 1
Humidity Sensor	Days / Hours / Single Use	< 1	8	8	23 / 1 / 1 / 1
Shock Sensor	Days / Hours / Single Use	30	8	240	671 / 2 / 1 / 1
Tilt sensor	Days / Hours / Single Use	1-100	8	400	1118 / 3 / 1 / 1
Vibration Sensor	Days / Hours / Single Use	30	8	240	671 / 2 / 1 / 1
Light Sensor	Days / Hours / Single Use	< 1	16	8	23 / 1 / 1 / 1
Accelerometer	Days / Hours / Single Use	30	8	240	671 / 2 / 1 / 1
Wine quality sensor	Days / Hours / Single Use	18.5	8	148	414 / 2 / 1 / 1
Gas identification	depends upon use case	25	16	200	559 / 2 / 1 / 1
Pressure Sensor	Days / Hours / Single Use	< 1	8	8	23 / 1 / 1 / 1
Trace Metal Sensor	Single Use	25	16	400	1 / 1 / 1 / 1

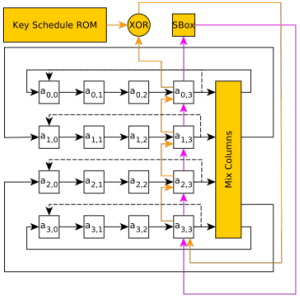
Datapaths w/ ROM Optimization



(a) Simon



(b) PRESENT



(c) AES-128

New Problem: Physical Vulnerability!

. . . Further Obfuscation?

- Prevent physically being able to read keys from ROM by scrambling them when programming to the ROM
- ROM contents are read into arrays of "Camouflage cells", whose contents are passed into `XOR` or `XNOR` with one input tied high
  - Buses are multiplexed by subround key mux - output is unscrambled subround key

Final Design Benchmarks

Latency (Cycles)		Area		Power		Energy	
	Cipher	mm^2	% Δ	mW	% Δ	μJ	% Δ
512	AES-128	4.31	-6.7	1.67	-20.4	85.3	-20.4
527	PRESENT-80	2.63	23.1	.95	-8.2	50.14	-8.2
527	PRESENT-128	2.71	-3.5	.97	-28.3	51.13	-28.3
32	Simon32-64	1.31	-10.3	.64	-11.3	2.06	-11.3
36	Simon48-72	1.54	-26.5	.84	-21.5	3.03	-21.5
36	Simon48-96	1.6	-26.1	.86	-21.3	3.1	-21.3
42	Simon64-96	1.88	-32.4	1.03	-26.5	4.35	-26.5
44	Simon64-128	2.	-30.6	1.05	-26.7	4.64	-26.7
52	Simon96-96	2.45	-38.8	1.29	-36.4	6.7	-36.4
54	Simon96-144	2.71	-32.2	1.35	-33.2	7.3	-33.2
68	Simon128-128	3.52	-33.4	1.58	-39.8	10.75	-39.8
69	Simon128-192	3.09	-41.6	1.42	-45.7	9.83	-45.7
72	Simon128-256	4.08	-25.3	1.7	-36.7	12.25	-36.7

The end!

If you want to read the papers this presentation was generated from, they were written by Prof. Kumar (the one who teaches 411!) and his grad students.

- Exploiting Short Application Lifetimes for Low Cost Hardware Encryption in Flexible Electronics, DATE, 2023
- FlexiCores: Low Footprint, High Yield, Field-Reprogrammable Flexible Microprocessors, ISCA, 2022